Speech Recognition (DSAI 456)

Lecture 5: Viterbi and Gaussian Mixture Models

Prof. Mohamed Ghalwash
mghalwash@zewailcity.edu.eg

Zewail City University

Lecture 5, Monday 26 October 2026.

Agenda

Objectives: by the end of this lecture you should be able to find the best alignment with the Viterbi algorithm and explain how it differs from the forward algorithm, use Viterbi for forced alignment, explain how a Gaussian mixture scores a continuous feature vector, fit a mixture with EM, and describe how GMMs and HMMs are trained together.

- **Viterbi**: the best alignment instead of all of them
- **Gaussian mixture models**: where $b_j(x_t)$ comes from
- **Putting it together**: training a GMM-HMM, and its limits

Part 1

The Viterbi Algorithm

The best alignment instead of all of them

From "How Likely?" to "Which One?"

The race from earlier today listed the three alignments of *salaam* that end in /a/:

Alignment	Probability
s | a a a	**0.0512**
s s | a a	0.0064
s s s | a	0.0128
Sum $= \alpha_4(a)$ (forward)	0.0704

From "How Likely?" to "Which One?"

The race from earlier today listed the three alignments of *salaam* that end in /a/:

Alignment	Probability
s | a a a	**0.0512**
s s | a a	0.0064
s s s | a	0.0128
Sum $= \alpha_4(a)$ (forward)	0.0704

- Forward tells us **how likely** the word is. To **transcribe**, or to find **where each phone is**, we need the top row: the single **best** alignment.

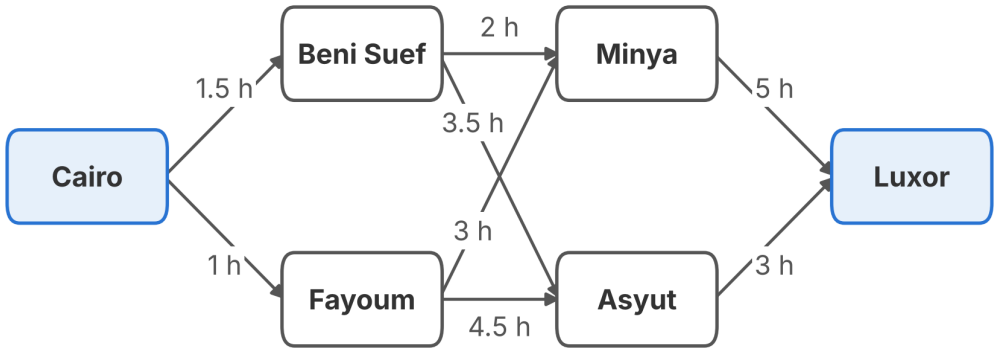
From "How Likely?" to "Which One?"

The race from earlier today listed the three alignments of *salaam* that end in /a/:

Alignment	Probability
s | a a a	**0.0512**
s s | a a	0.0064
s s s | a	0.0128
Sum $= \alpha_4(a)$ (forward)	0.0704

- Forward tells us **how likely** the word is. To **transcribe**, or to find **where each phone is**, we need the top row: the single **best** alignment.
- Listing is hopeless (10^{86} alignments for one sentence). Idea: in each trellis cell, **keep only the best incoming path** and **remember where it came from**. Is that safe?

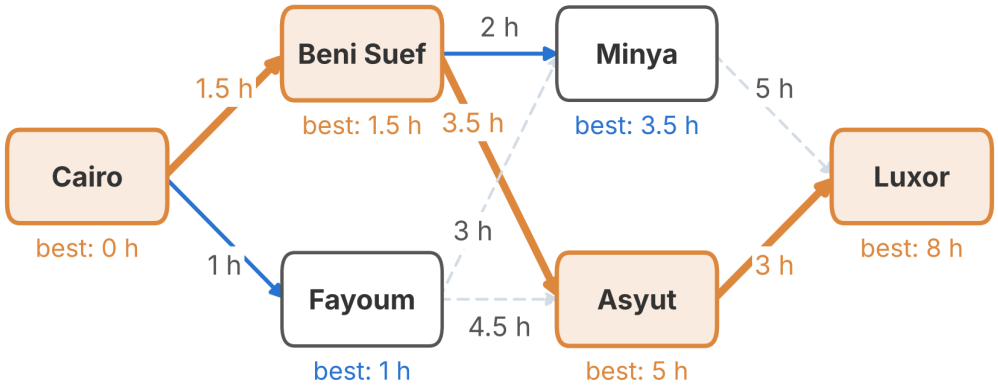
Activity 1: The Fastest Route to Luxor (3 min)



Travel times in hours (invented). Find the **fastest** route from Cairo to Luxor.

Rule: each city may hold **one number only**: the fastest time to reach it so far. Work from left to right.

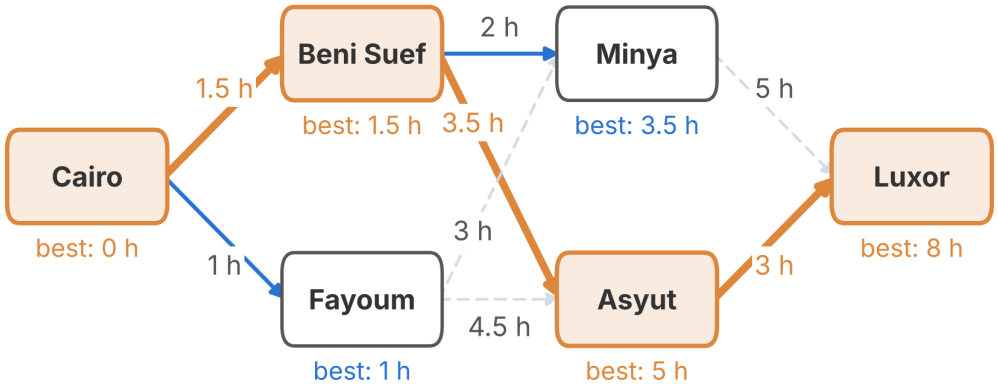
Activity 1: The Fastest Route to Luxor (3 min)



8 h, via Beni Suef and Asyut. Minya keeps only its best arrival (3.5 h) and forgets the slower one (4 h).

A **greedy** driver, who always takes the fastest next road, goes via Fayoum and Minya: **9 h**.

Activity 1: The Fastest Route to Luxor (3 min)



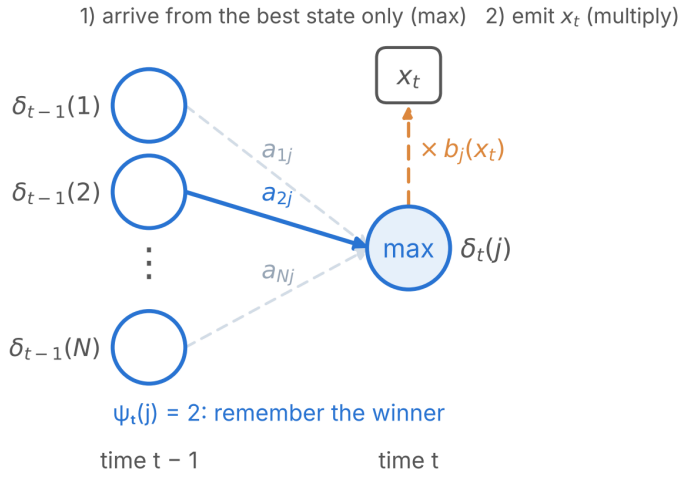
This is Viterbi.

Road trip	Viterbi
city	state
column of cities	frame t
hours	$-\log$ probability
fastest	most likely
best time at a city	$\delta_t(j)$
where you came from	$\psi_t(j)$

Why is forgetting safe? If the fastest trip to Luxor passes through Asyut, its first part must be the fastest way to Asyut. Otherwise, swap in the faster one and the whole trip gets faster.

The Viterbi Algorithm

The road trip, in general. $\delta_t(j)$ is the probability of the **best** path that explains $x_1, \ldots, x_t$ and ends in state j at time t.

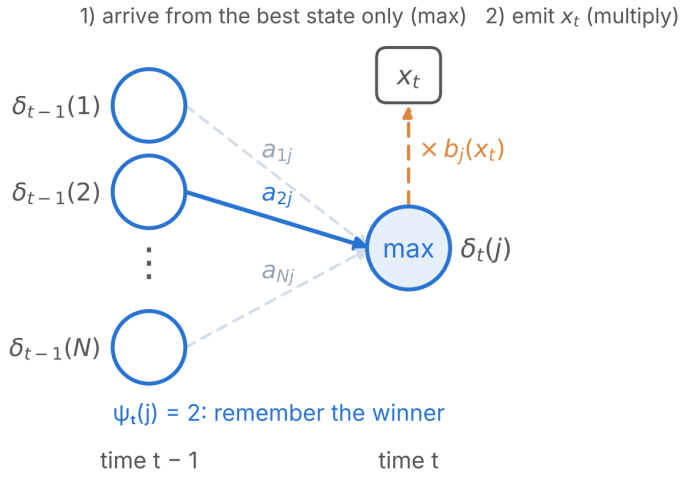


> **Note**
> In ASR the best path gives two things at once: **decoding** (which words) and **alignment** (which frames belong to which phone).

The Viterbi Algorithm

The road trip, in general. $\delta_t(j)$ is the probability of the **best** path that explains $x_1, \ldots, x_t$ and ends in state j at time t.

- **Initialize**: $\delta_1(j) = \pi_j \, b_j(x_1)$ · start: Cairo, 0 h

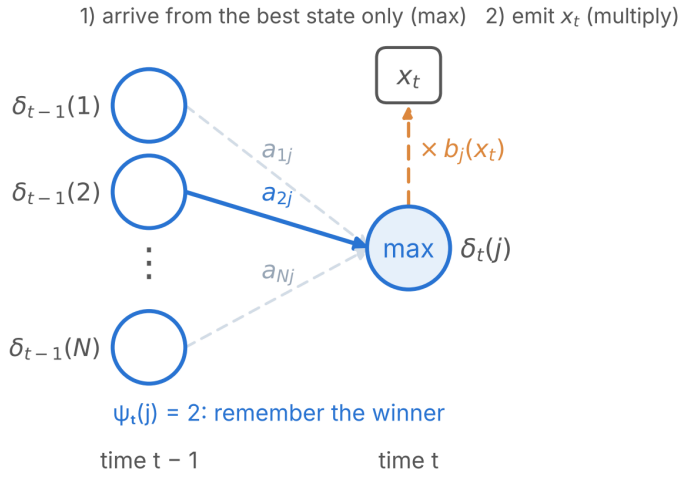


> **Note**
> In ASR the best path gives two things at once: **decoding** (which words) and **alignment** (which frames belong to which phone).

The Viterbi Algorithm

The road trip, in general. $\delta_t(j)$ is the probability of the **best** path that explains $x_1, \ldots, x_t$ and ends in state j at time t.

- **Initialize**: $\delta_1(j) = \pi_j \, b_j(x_1)$ · start: Cairo, 0 h
- **Recurse**: $\delta_t(j) = \left[\max_i \delta_{t-1}(i) \, a_{ij} \right] b_j(x_t)$ · Minya: min(1.5 + 2, 1 + 3) = 3.5 h

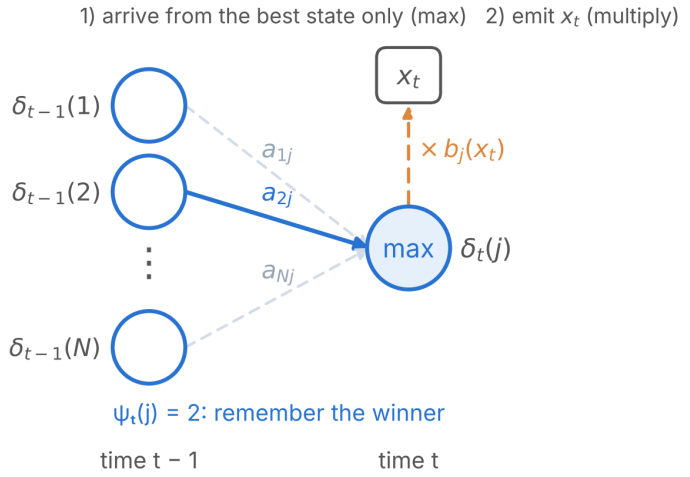


> **Note**
> In ASR the best path gives two things at once: **decoding** (which words) and **alignment** (which frames belong to which phone).

The Viterbi Algorithm

The road trip, in general. $\delta_t(j)$ is the probability of the **best** path that explains $x_1, \ldots, x_t$ and ends in state j at time t.

- **Initialize**: $\delta_1(j) = \pi_j \, b_j(x_1)$ · start: Cairo, 0 h
- **Recurse**: $\delta_t(j) = \left[\max_i \delta_{t-1}(i) \, a_{ij} \right] b_j(x_t)$ · Minya: min(1.5 + 2, 1 + 3) = 3.5 h
- **Remember**: $\psi_t(j) = \arg\max_i \; \delta_{t-1}(i) \, a_{ij}$ · Minya came from Beni Suef

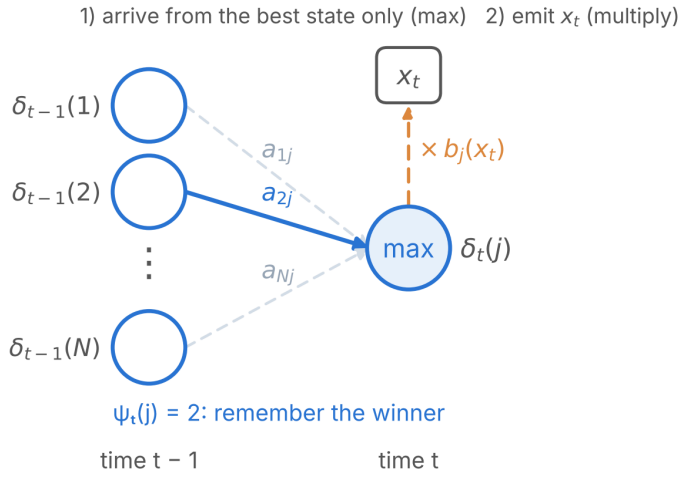


> **Note**
> In ASR the best path gives two things at once: **decoding** (which words) and **alignment** (which frames belong to which phone).

The Viterbi Algorithm

The road trip, in general. $\delta_t(j)$ is the probability of the **best** path that explains $x_1, \ldots, x_t$ and ends in state j at time t.

- **Initialize**: $\delta_1(j) = \pi_j \, b_j(x_1)$ · start: Cairo, 0 h
- **Recurse**: $\delta_t(j) = \left[\max_i \delta_{t-1}(i) \, a_{ij} \right] b_j(x_t)$ · Minya: min(1.5 + 2, 1 + 3) = 3.5 h
- **Remember**: $\psi_t(j) = \arg\max_i \; \delta_{t-1}(i) \, a_{ij}$ · Minya came from Beni Suef
- **Terminate**: $P^* = \delta_T(N)$, since a word must end in its last state · the goal is fixed: Luxor, 8 h

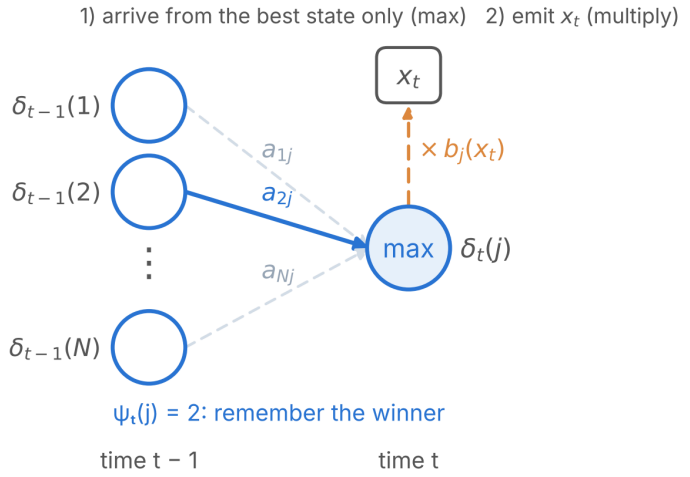


> **Note**
> In ASR the best path gives two things at once: **decoding** (which words) and **alignment** (which frames belong to which phone).

The Viterbi Algorithm

The road trip, in general. $\delta_t(j)$ is the probability of the **best** path that explains $x_1, \ldots, x_t$ and ends in state j at time t.

- **Initialize**: $\delta_1(j) = \pi_j \, b_j(x_1)$ · start: Cairo, 0 h
- **Recurse**: $\delta_t(j) = \left[\max_i \delta_{t-1}(i) \, a_{ij} \right] b_j(x_t)$ · Minya: min(1.5 + 2, 1 + 3) = 3.5 h
- **Remember**: $\psi_t(j) = \arg\max_i \; \delta_{t-1}(i) \, a_{ij}$ · Minya came from Beni Suef
- **Terminate**: $P^* = \delta_T(N)$, since a word must end in its last state · the goal is fixed: Luxor, 8 h
- **Backtrace**: $q_t^* = \psi_{t+1}(q_{t+1}^*)$ for $t = T-1, \ldots, 1$ · Luxor ← Asyut ← Beni Suef ← Cairo

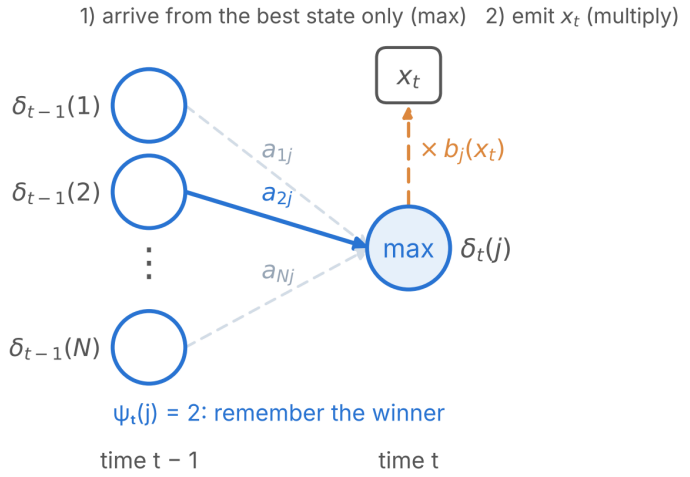


> **Note**
> In ASR the best path gives two things at once: **decoding** (which words) and **alignment** (which frames belong to which phone).

The Viterbi Algorithm

The road trip, in general. $\delta_t(j)$ is the probability of the **best** path that explains $x_1, \ldots, x_t$ and ends in state j at time t.

- **Initialize**: $\delta_1(j) = \pi_j \, b_j(x_1)$ · start: Cairo, 0 h
- **Recurse**: $\delta_t(j) = \left[\max_i \delta_{t-1}(i) \, a_{ij} \right] b_j(x_t)$ · Minya: min(1.5 + 2, 1 + 3) = 3.5 h
- **Remember**: $\psi_t(j) = \arg\max_i \; \delta_{t-1}(i) \, a_{ij}$ · Minya came from Beni Suef
- **Terminate**: $P^* = \delta_T(N)$, since a word must end in its last state · the goal is fixed: Luxor, 8 h
- **Backtrace**: $q_t^* = \psi_{t+1}(q_{t+1}^*)$ for $t = T-1, \ldots, 1$ · Luxor ← Asyut ← Beni Suef ← Cairo
- **Cost**: $O(N^2 T)$, like forward. In log space max stays max, so no logsumexp is needed: $\log \delta_t(j) = \log b_j(x_t) + \max_i \left(\log \delta_{t-1}(i) + \log a_{ij} \right)$

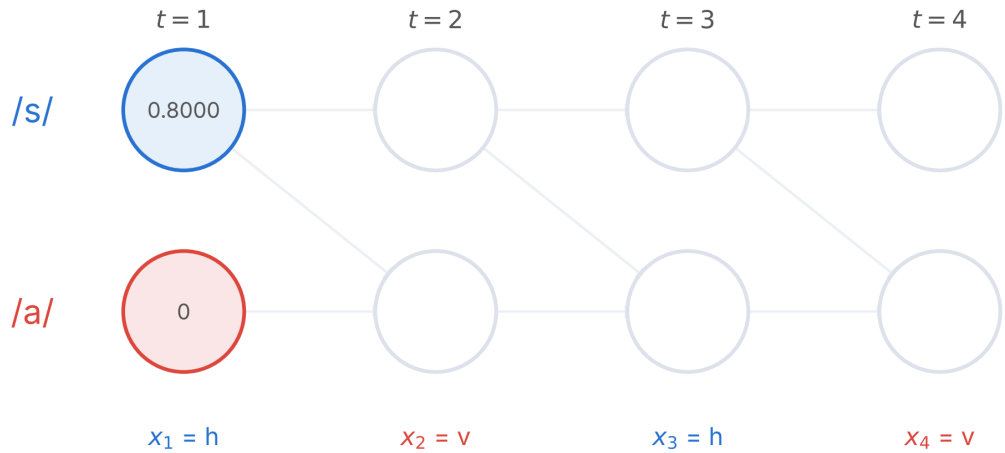


> **Note**
> In ASR the best path gives two things at once: **decoding** (which words) and **alignment** (which frames belong to which phone).

Activity 2: Viterbi by Hand (8 min)

Same *salaam* model, same $X = (h, v, h, v)$. Fill the trellis with **max instead of sum**, and draw an arrow from each cell to the predecessor that won.

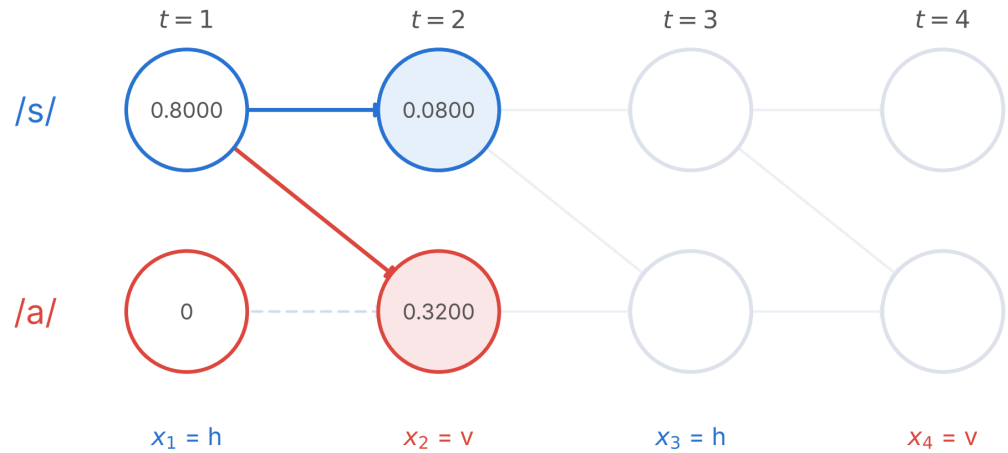
$t = 1$: nothing to choose yet: $\delta_1(s) = 1 \cdot 0.8 = 0.8,\ \delta_1(a) = 0.$



Activity 2: Viterbi by Hand (8 min)

Same *salaam* model, same $X = (h, v, h, v)$. Fill the trellis with **max instead of sum**, and draw an arrow from each cell to the predecessor that won.

$t = 2$ ($x_2 = v$): $\delta_2(s) = 0.8 \cdot 0.5 \cdot 0.2 = 0.08$, $\delta_2(a) = \max(0.8 \cdot 0.5,\ 0 \cdot 1) \cdot 0.8 = 0.32$, from /s/.

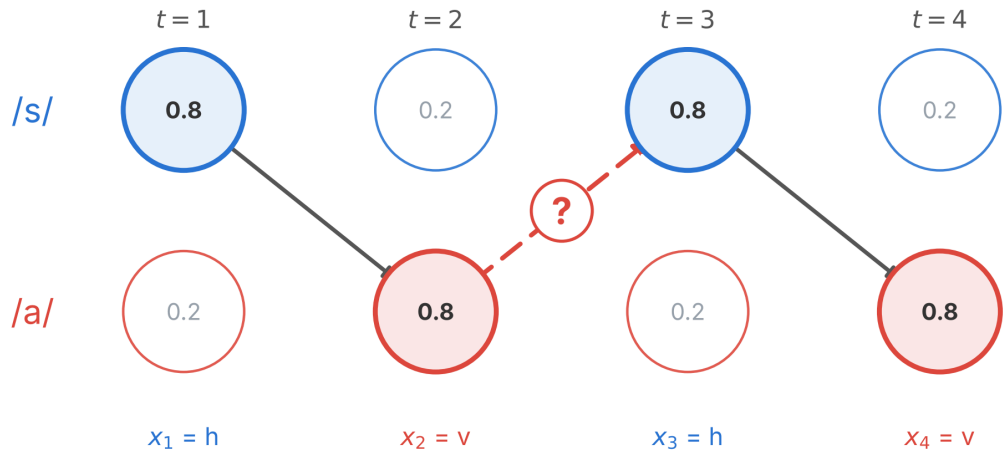


Same as α_2: /a/ had only one real way in.

Activity 2: Viterbi by Hand (8 min)

Same *salaam* model, same $X = (h, v, h, v)$. Fill the trellis with **max instead of sum**, and draw an arrow from each cell to the predecessor that won.

Shortcut? Pick each frame's best emission on its own: h → /s/, v → /a/. That gives **s a s a**. **Hands up**: is that a possible path?

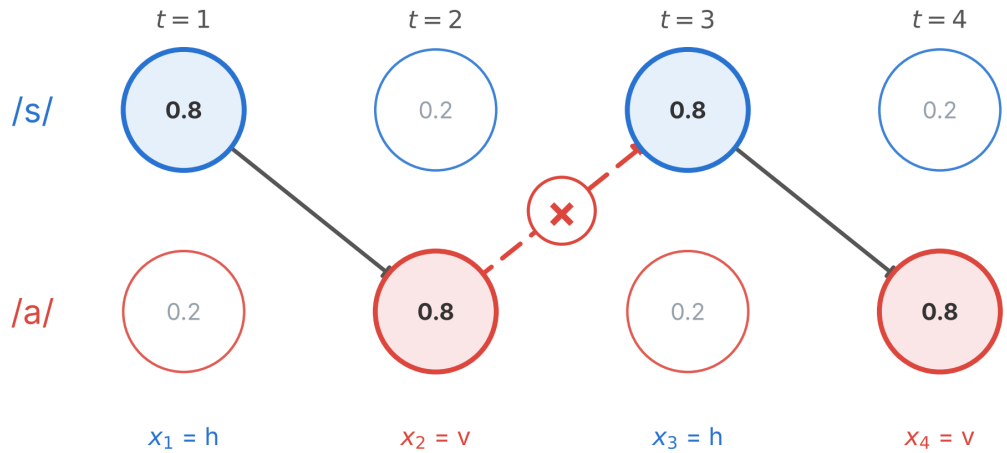


The circles show only the emission probabilities $b_j(x_t)$.

Activity 2: Viterbi by Hand (8 min)

Same *salaam* model, same $X = (h, v, h, v)$. Fill the trellis with **max instead of sum**, and draw an arrow from each cell to the predecessor that won.

No. s a s a needs a jump from /a/ back to /s/, and that transition has probability 0.

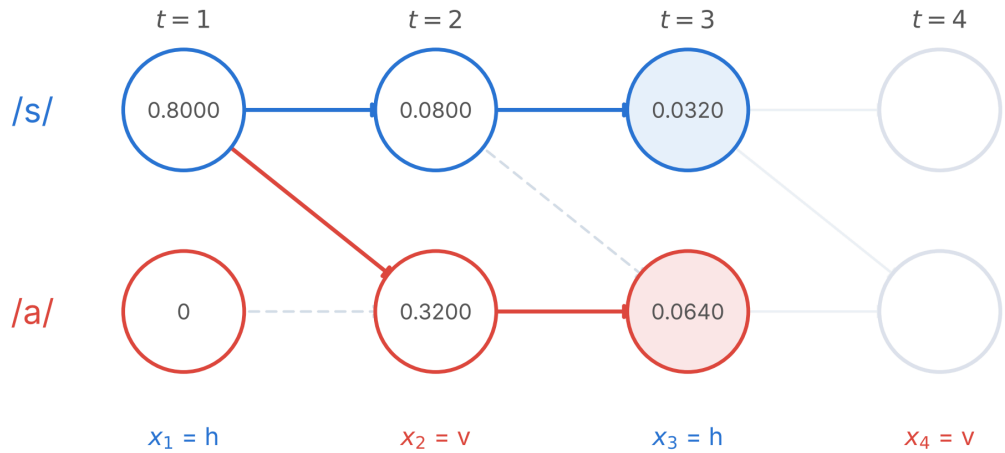


Looking at one frame at a time ignores the transitions. Viterbi uses both.

Activity 2: Viterbi by Hand (8 min)

Same *salaam* model, same $X = (h, v, h, v)$. Fill the trellis with **max instead of sum**, and draw an arrow from each cell to the predecessor that won.

$t = 3$ ($x_3 = h$): $\delta_3(s) = 0.032,\ \ \delta_3(a) = \max(0.08 \cdot 0.5,\ 0.32 \cdot 1) \cdot 0.2 = 0.064$, from /a/.

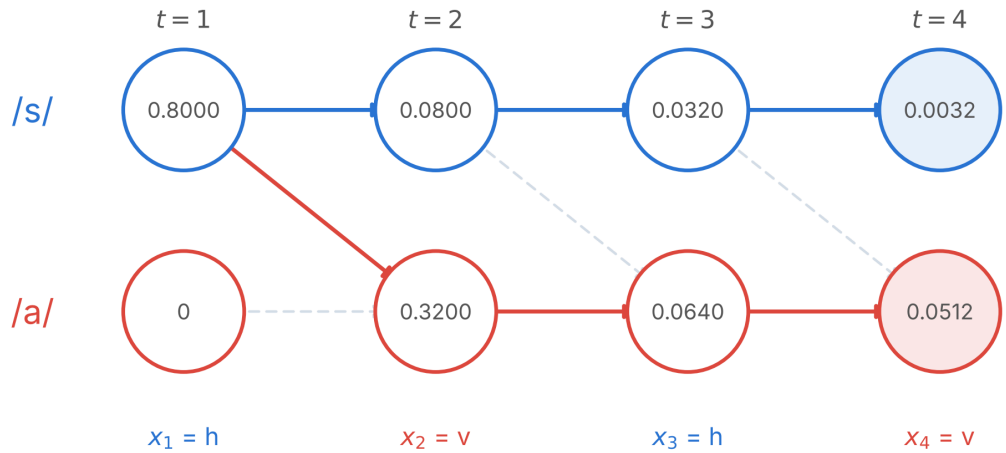


The h favours /s/, but the best path stays in /a/ ($0.064 > 0.032$). Forward added both arrows into /a/ ($\alpha_3(a) = 0.072$); Viterbi keeps the larger.

Activity 2: Viterbi by Hand (8 min)

Same *salaam* model, same $X = (h, v, h, v)$. Fill the trellis with **max instead of sum**, and draw an arrow from each cell to the predecessor that won.

$t = 4$ ($x_4 = v$): $\delta_4(s) = 0.0032,\ \delta_4(a) = \max(0.032 \cdot 0.5,\ 0.064 \cdot 1) \cdot 0.8 = 0.0512$, from /a/.

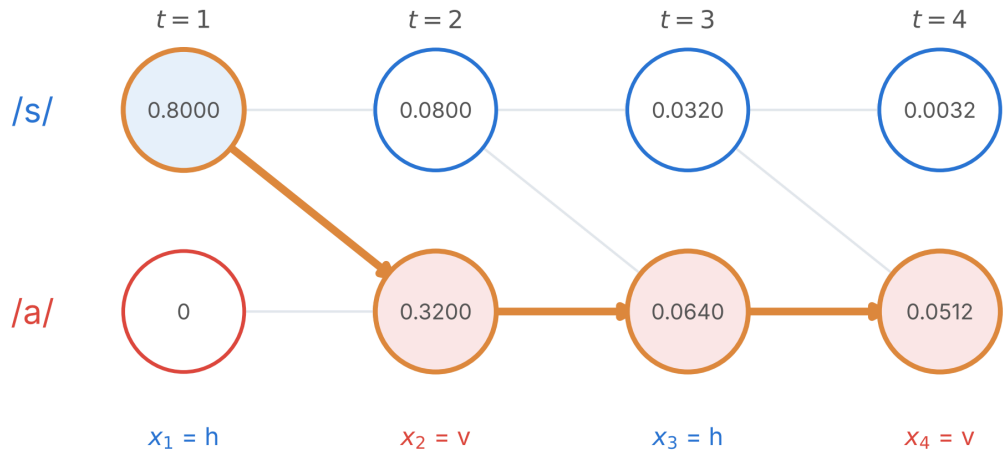


Solid arrow: the winning predecessor. Dashed: the loser.

Activity 2: Viterbi by Hand (8 min)

Same *salaam* model, same $X = (h, v, h, v)$. Fill the trellis with **max instead of sum**, and draw an arrow from each cell to the predecessor that won.

Backtrace: the word must end in /a/, so start at $\delta_4(a)$ and follow the arrows back: $\psi_4(a) = a$, $\psi_3(a) = a$, $\psi_2(a) = s$.



The answer is **s | a a a** with $P^* = 0.0512$: the top row of the race table, found without listing anything.

Activity 3: Bug Hunt (5 min)

`delta[t, j]` and `psi[t, j]` (the best predecessor of state `j` at time `t`) are filled for *salaam*. States: 0 = /s/, 1 = /a/; the word ends in the last state, `N − 1`. **Only one backtrace is correct.** Vote in Slido.

```python
# A
path = [N - 1]
for t in range(T - 1, 0, -1):
    path.insert(0, psi[t, path[0]])
```

```python
# B
path = [N - 1]
for t in range(T - 1, 0, -1):
    path.insert(0, psi[t - 1, path[0]])
```

```python
# C
path = [B[:, x[t]].argmax()
        for t in range(T)]
```

```python
# D
path = [0]
for t in range(1, T):
    path.append(psi[t, path[-1]])
```

Activity 3: Bug Hunt (5 min)

`delta[t, j]` and `psi[t, j]` (the best predecessor of state `j` at time `t`) are filled for *salaam*. States: 0 = /s/, 1 = /a/; the word ends in the last state, `N − 1`. **Only one backtrace is correct.** Vote in Slido.

```python
# A
path = [N - 1]
for t in range(T - 1, 0, -1):
    path.insert(0, psi[t, path[0]])
```

```python
# B
path = [N - 1]
for t in range(T - 1, 0, -1):
    path.insert(0, psi[t - 1, path[0]])
```

```python
# C
path = [B[:, x[t]].argmax()
        for t in range(T)]
```

```python
# D
path = [0]
for t in range(1, T):
    path.append(psi[t, path[-1]])
```

A is correct: s a a a (0.0512). **B** reads ψ one column too early and returns s s a a, a real path (0.0064), so the bug hides. **C** ignores the transitions: s a s a, the greedy trap (probability 0). **D** follows the arrows forwards from /s/: s s s s, which never reaches /a/.

Sum vs. Max on *salaam*

t	$\alpha_t(s)$	$\delta_t(s)$	$\alpha_t(a)$	$\delta_t(a)$
1	0.8	0.8	0	0
2	0.08	0.08	0.32	0.32
3	0.032	0.032	0.072	0.064
4	0.0032	0.0032	**0.0704**	**0.0512**

- Each δ is **one term** of the sum that makes α, so $\delta_t(j) \leq \alpha_t(j)$. They are equal while only one path has non-zero probability.
- Here the best path carries $0.0512/0.0704 = 73\%$ of the total.

Activity 4: Best Path or Total? (4 min)

Neither algorithm picks a word: each one gives a word a **score**. To recognize a word, we score the audio with **every word's HMM** and keep the highest score. Which score: the **total** over all alignments, $P(X \mid \text{word})$ (forward), or the **best** alignment only, P^* (Viterbi)?

Word A has **one** alignment, with probability 0.010. Word B has **ten** alignments, with 0.002 each. **Poll**: which word wins with each score?

Activity 4: Best Path or Total? (4 min)

Neither algorithm picks a word: each one gives a word a **score**. To recognize a word, we score the audio with **every word's HMM** and keep the highest score. Which score: the **total** over all alignments, $P(X \mid \text{word})$ (forward), or the **best** alignment only, P^* (Viterbi)?

Word A has **one** alignment, with probability 0.010. Word B has **ten** alignments, with 0.002 each. **Poll**: which word wins with each score?



Score	Word A	Word B	Winner
Best path (Viterbi)	**0.010**	0.002	**A**
Total (forward)	0.010	**0.020**	**B**

The two scores **rank the words differently** when one word spreads its probability over many alignments. The total is the true $P(X \mid \text{word})$; the best path is a fast stand-in that usually ranks the same way (73% of the total on *salaam*).

Activity 4: Best Path or Total? (4 min)

Neither algorithm picks a word: each one gives a word a **score**. To recognize a word, we score the audio with **every word's HMM** and keep the highest score. Which score: the **total** over all alignments, $P(X \mid \text{word})$ (forward), or the **best** alignment only, P^* (Viterbi)?

Word A has **one** alignment, with probability 0.010. Word B has **ten** alignments, with 0.002 each. **Poll**: which word wins with each score?



Score	Word A	Word B	Winner
Best path (Viterbi)	**0.010**	0.002	**A**
Total (forward)	0.010	**0.020**	**B**

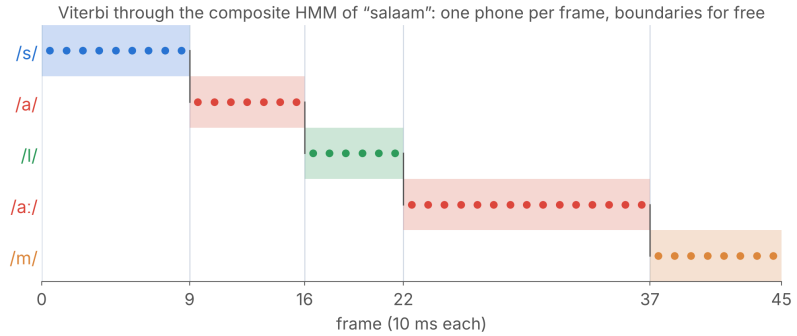
The two scores **rank the words differently** when one word spreads its probability over many alignments. The total is the true $P(X \mid \text{word})$; the best path is a fast stand-in that usually ranks the same way (73% of the total on *salaam*).

Try it in **Trellis Lab**: find two words that the two scores rank differently.

Forced Alignment

When the transcript is **known**, Viterbi tells us **where** each phone is.

1. Build one long left-to-right HMM for the sentence: words → phones (lexicon) → 3 states per phone.
2. Run Viterbi over the whole utterance.
3. Read the best path: every frame gets a phone, and every phone gets a start and end frame.



Last week we disagreed on where /l/ starts; Viterbi commits to **one** answer. These frame labels are what a neural frame classifier will need for training.

Part 2

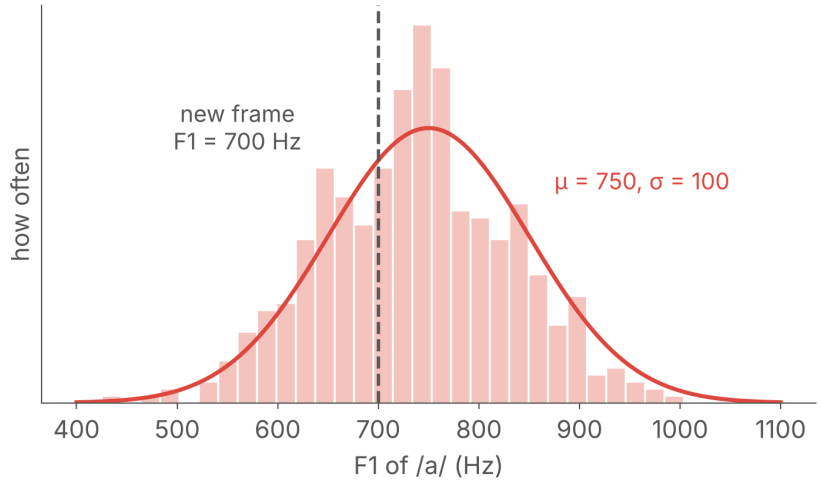
Gaussian Mixture Models

Where $b_j(x_t)$ comes from

From Symbols to Vectors

- In the *salaam* model, B was a **table**: /s/ emits h with probability 0.8.
- A real frame is a vector of **39 real numbers** (13 MFCC + Δ + ΔΔ). A table cannot list every possible vector, so $b_j(x_t)$ must be a **density**: a function that scores any vector. (A density can be larger than 1; it is not a probability.)
- We build it in steps, starting with **one** feature we already know: F1, the first formant (Lecture 2). F1 is not one of the 39, but it is easy to picture, and each coefficient works the same way.

Question: a new frame has F1 = 700 Hz. Does it look like /a/? How much?



One Bump: the Gaussian

A bell curve needs only a centre μ and a width σ:

$$\mathcal{N}(x;\mu,\sigma) = \frac{1}{\sqrt{2\pi}\,\sigma}\,\exp\!\Big(-\frac{z^2}{2}\Big), \qquad z = \frac{x-\mu}{\sigma}$$

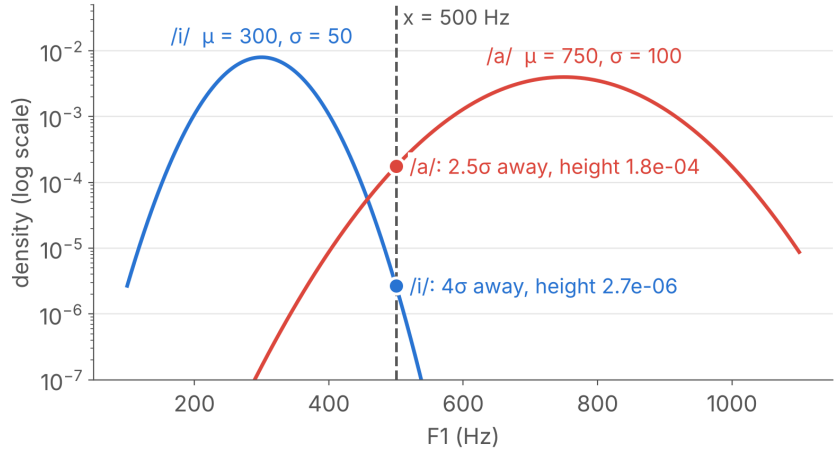
Piece	What it does
$z = (x-\mu)/\sigma$	how many **σ-widths** x is from the centre
$\exp(-z^2/2)$	turns that distance into a score: 1 at the centre, falling fast
$1/(\sqrt{2\pi}\,\sigma)$	makes the area under the curve equal 1

Activity 5: Which Vowel? (3 min)

Poll: /i/ has $\mu = 300$ Hz, $\sigma = 50$ Hz. /a/ has $\mu = 750$ Hz, $\sigma = 100$ Hz. A frame has F1 = **500 Hz**. Which vowel fits better? (Hint: 500 is closer to /i/ in Hz.)

Activity 5: Which Vowel? (3 min)

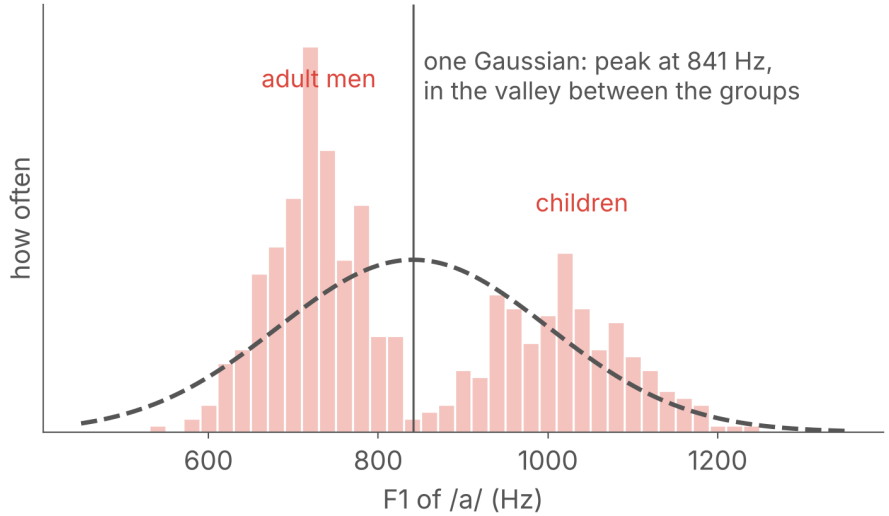
Poll: /i/ has $\mu = 300$ Hz, $\sigma = 50$ Hz. /a/ has $\mu = 750$ Hz, $\sigma = 100$ Hz. A frame has F1 = **500 Hz**. Which vowel fits better? (Hint: 500 is closer to /i/ in Hz.)



/i/: $(500 - 300)/50 = 4\sigma$ away, height $\approx 2.7 \times 10^{-6}$. /a/: $(500 - 750)/100 = -2.5\sigma$ away, height $\approx 1.8 \times 10^{-4}$, about **65 times higher**. **/a/ wins**: distance is measured in σ, not in Hz.

One Bump Is Not Enough

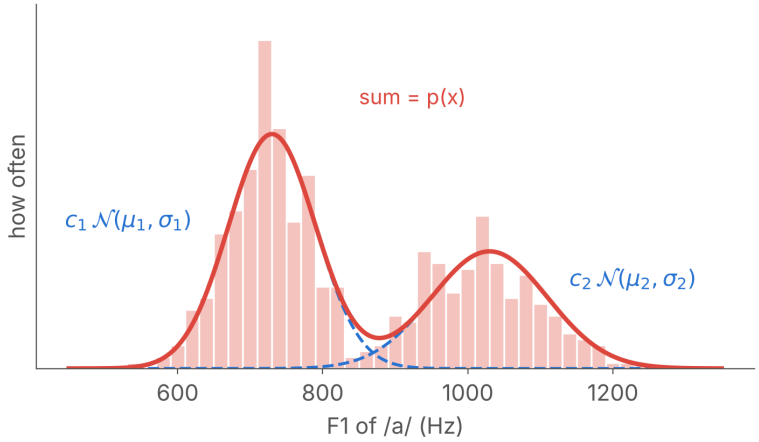
The same vowel /a/, from adult men (~730 Hz) and children (~1030 Hz), makes **two bumps**.



One Gaussian puts its peak **in the valley**, where there is almost no data, and scores real /a/ frames poorly.

A Mixture of Bumps

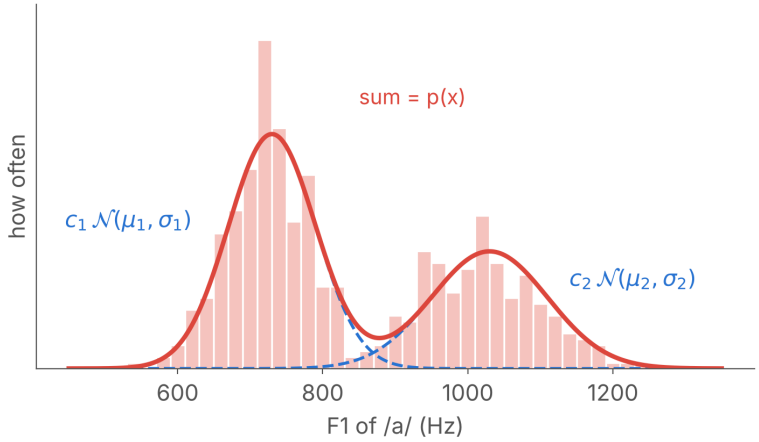
The fix is to **add** bumps, each with a weight:



$$p(x) = \sum_{m=1}^{M} c_m\,\mathcal{N}(x; \mu_m, \sigma_m), \qquad \sum_m c_m = 1$$

A Mixture of Bumps

The fix is to **add** bumps, each with a weight:



$$p(x) = \sum_{m=1}^{M} c_m\,\mathcal{N}(x; \mu_m, \sigma_m), \qquad \sum_m c_m = 1$$

Fitting One Bump Is Easy

If all the points $x_1, \ldots, x_T$ come from **one** bump, fitting it means two averages:

$$\mu = \frac{1}{T} \sum_t x_t \qquad\qquad \sigma^2 = \frac{1}{T} \sum_t (x_t - \mu)^2$$

- μ: the **average** of the points.
- σ^2: the **average squared distance** from μ.

Fitting One Bump Is Easy

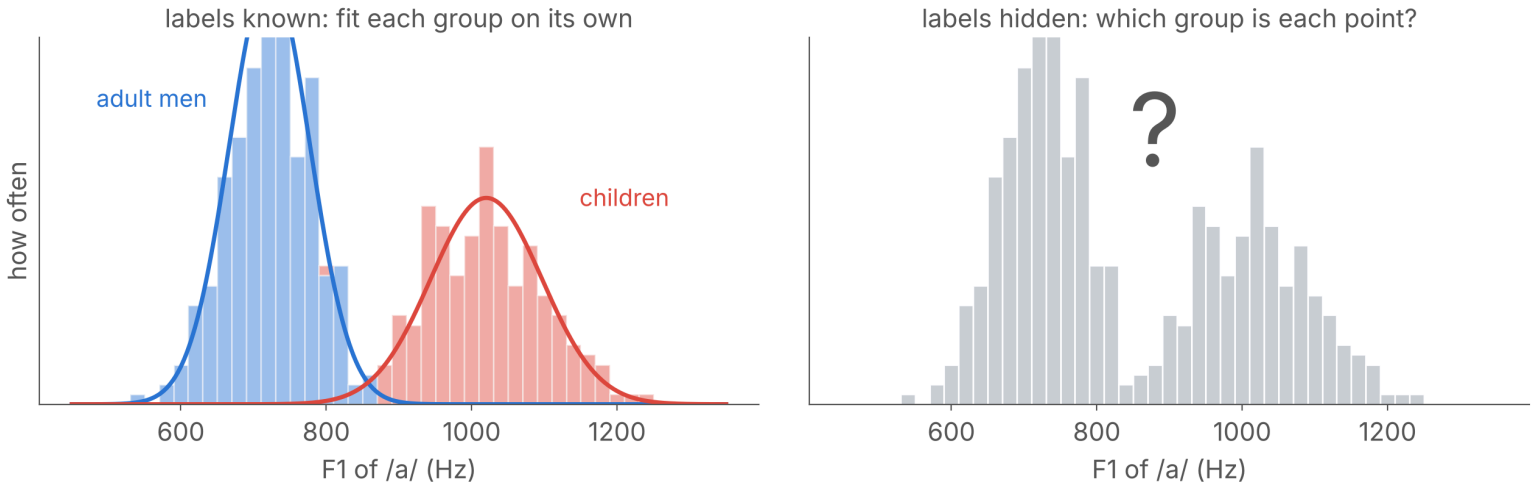
If all the points $x_1, \ldots, x_T$ come from **one** bump, fitting it means two averages:

$$\mu = \frac{1}{T}\sum_t x_t \qquad\qquad \sigma^2 = \frac{1}{T}\sum_t (x_t - \mu)^2$$

- μ: the **average** of the points.
- σ^2: the **average squared distance** from μ.

With **two** bumps, the question is: which points do we average for which bump?

Two Bumps: Who Belongs Where?



- **Left**: if every point had a label (man or child), we would fit each bump on its own points, exactly as on the previous slide.
- **Right**: in real data the labels are **hidden**. Nobody tells us which group a frame came from.

Assignment 5: Viterbi and GMMs

1. **Implement** Viterbi in NumPy, in the **log domain**, for the *salaam* model. Reproduce $\delta_4(a) = 0.0512$ and the path s | a a a.
2. **Verify** with `hmmlearn.hmm.CategoricalHMM.decode(..., algorithm="viterbi")`, and against brute force by listing every alignment. Would brute force catch bug B from Activity 3?

> **Tip**
> Report the hardware you used and the wall-clock time for any timing or fitting experiment.

Learn More

Course Homepage · Rabiner (1989): A Tutorial on Hidden Markov Models · J&M: Appendix A, Hidden Markov Models · scikit-learn: Gaussian mixture models · Montreal Forced Aligner · hmmlearn